Agile Planning and Tracking of Software Projects Using the State-Space Approach

Manoj Kumar Tyagi[1(✉)], Dheerendra Kumar Tyagi[1], Lalit Kumar Tyagi[2], Neha Tyagi[1], Durgesh Kumar[3], and Ajay Sikandar[3]

[1] Department of Information Technology, G L Bajaj Institute of Technology and Management, Greater Noida, India
manojankitsuhani@gmail.com, tyagidk80@gmail.com, neha.tyagi@glbitm.ac.in

[2] Department of CSE-Artificial Intelligence, G L Bajaj Institute of Technology and Management, Greater Noida, India
tyagilalit70@yahoo.co.in

[3] Department of Computer Science and Engineering, G L Bajaj Institute of Technology and Management, Greater Noida, India
durgesh.durge@gmail.com, ajay.sikandar@gmail.com

Abstract. All projects are required to be tracked and reported their status information to the management for controlling their execution, and have limited resources in terms of budget and time, so agile projects are. Their tracking is an overhead for the software projects as the tracking is a management activity, not a technical one, for the development of the software product. Various tracking techniques have been developed for tracking of agile projects. We have classified them as heavy-weight and light-weight, in terms of the information maintained, or not, about which phase of the project lifecycle a partially completed task belongs to. Light-weight techniques consider the tasks which have been completed but do not consider the partially completed tasks; while heavy-weight ones consider the tasks which have been completed as well as the state information about which phase of the project lifecycle the partially completed tasks belong to. Light-weight techniques are not effective due to not considering the partially completed tasks and heavy-weight ones are not utilizing the resources efficiently. In this paper, a light-weight tracking technique is designed using the state-space approach, which considers both the completed tasks and the partially completed tasks but without maintaining the state information for the partially completed tasks in order to tracking the projects effectively and utilizing the resources efficiently. The application of the technique, using Monte-Carlo simulation, is demonstrated by tracking a project which was carried out with the agile software development philosophy.

Keywords: Agile software project · State-space approach · Monte-Carlo simulation · Team productivity · Organizational productivity

© Springer Nature Singapore Pte Ltd. 2021
D. Garg et al. (Eds.): IACC 2020, CCIS 1368, pp. 355–371, 2021.
https://doi.org/10.1007/978-981-16-0404-1_26

1 Introduction

The tracking of a project status and communicating it to project manager(s) is an essential activity during a project execution. The project status is needed to be tracked and communicated so that prospective problems can be found before their occurrence. Project manager decides about a project status whether it is acceptable or some actions are needed to control the project execution. However, the tracking being a management activity, it is not directly involved in the development of a software product and can be considered as an overhead over the software projects having limited budget and time. The overhead is measured in terms of the information maintained by a tracking technique for the project related tasks. Therefore, there is a need of light-weight tracking techniques in order to track the projects efficiently.

Earned value tracking and reporting [1] has been made a compulsory tool to be used for carrying out government projects [2]. However, agile communities prefer other tools as well. For example, scrum community uses burn down charts [3], feature driven development practitioners use cumulative flow diagrams ([4–6]), xtreme programming community uses stories completed and tests passed [7].

During a project execution, there are three types of tasks: tasks whose implementation not yet started, tasks whose implementation have been started but not yet consummated (partially completed tasks) and tasks whose implementation have been consummated (completed tasks), to be considered for the project status evaluation and reporting. The amount of work left during project execution and as per plan is reported by burn down chart, and stories completed and tests passed. Both burn down chart, and stories completed and tests passed consider only those tasks for the evaluation of the project status which have been completed, but not the partially completed tasks. All the tasks along with their state information (which phase of the project lifecycle a task belongs to) are reported during project execution, but not as per plan, by cumulative flow diagrams; while the line of balance method is capable of reporting not only during project execution but also as per plan [8].

The tracking techniques are classified as heavy-weight or light-weight based on whether the state information of partially completed tasks is maintained by a technique, or not, respectively. Generally, the project execution goes smoothly means that the status reported by a technique is most of the time acceptable without any further action taken for the project being executed. Using the heavy-weight tracking techniques for projects imposes more overhead (resources consumed for maintaining the state information of partially completed tasks during project execution) which outweighs the benefits (better project visibility to decide some actions) of them. Also, as per the agile manifesto, the value has been given to individuals and interactions over processes and tools [9]. However, it does not mean that tools should not be used, but these must be light-weight.

In this paper, a light-weight tracking technique has been designed using the state-space approach in [10], which considers both the completed tasks and the partially completed tasks but without maintaining the state information for the partially completed tasks in order to track the projects effectively and efficiently. In line with the state-space approach, the technique has two components the process component and

the observation component and it is used for tracking a project in its plan space and execution space. The process component determines the project state which subsequently is used by the observation component for determining the project status. In order to generalize the technique, we use task as a representation of the project work.

The rest of the paper is structured as follows. Section 2 discusses the agile projects tracking techniques and the classification of them as the light-weight or heavy-weight. Section 3 discusses the state-space approach. Section 4 consists of design issues. Section 5 describes the tracking technique formulation. Section 6 consists of simulation study and results. Finally, we conclude in Sect. 7.

2 Agile Projects Tracking Techniques

Agile projects are categorized as to evolve the software product iteratively. With following a brief analysis phase for an iteration, the product is evolved by performing a sequence of activities: designing, coding, testing, and integrating; and the tasks planned for an iteration are independent from the other iterations. This process goes on iteratively until the product is not developed completely.

Scrum community uses burn down chart, burn up chart and a task board [11]. Burn down chart shows the tasks leftover to be completed and the rate of progress. The tasks being added or dropped to a project is represented by moving the curve upward or downward, respectively. Burn up chart shows the tasks completed and the rate of progress. By using the rate of progress, the project completion time can be predicted.

The task board is used for tracking and reporting the project status as well as a mechanism for a team to organize its work. During the life of a task, it has three states: awaiting (waiting for team members to get allotted), live (task implementation related activities are going on), and winded up (task is completed). Task implementation related activities (designing, coding, testing, and integrating) are represented using the corresponding cards. During the project execution, the cards are moved from one state (awaiting, live, and winded up) to other. A task is completed when all the cards are moved to the winded up state. The task board considers only those tasks which have been completed in order to report the project status as well as the rate of progress. The information about the tasks in the live state (i.e., partially completed tasks) has no significance as what matters for reporting is either the tasks completed or tasks leftover.

Feature driven development community favors for cumulative flow diagrams ([4], [5], [12]). The cumulative flow diagrams show the number of tasks in which phase (designing, coding, testing, and integrating) of the task implementation these belongs to during the project execution, but does not relate it to the project plan. However, the line of balance method does the same in relation to the project plan as well [8].

Earned value technique has been used in agile projects for reporting the project status [13]. The application of the earned value method for tracking of agile projects has recommended some changes at the fundamental concepts level [14]. In order to report the project status, this also considers only those tasks which have been completed.

According to the literature reviewed above, following observations are important to notice. Burn down chart, burn up chart, task board, and earned value method considers

only the tasks which have been completed, not the partially completed ones, for the tracking of projects. These do not need to maintain the state information for the partially completed tasks (to which phase of the task implementation a partially completed task belongs to). While the cumulative flow diagrams and the line of balance method consider the tasks which have been completed and the partially completed tasks along with their state information. The resources required for maintaining the state information corresponding to the partially completed tasks makes a tracking technique, as heavy-weight or light-weight, if it maintains, or not, respectively, the state information. Hence, Burn down chart, burn up chart, task board, and earned value method are termed as light-weight; while cumulative flow diagrams and line of balance method, as heavy-weight.

3 State-Space Approach

Modern control methods recommend using the state-space approach for designing a system. The state-space approach is used to design a system considering the time dimension and is capable of representing all the components of a system. The system to be designed using the state-space approach here is assumed to be a linear time invariant system. A linear time invariant system is one which can be represented using the first order differential equations in terms of the state variables representing the internal system behavior, external input variables and the noise, and does not vary with time. These state variables along with the external input variables and the noise can be used to determine the output variables. The detailed description about state-space approach can be found in [10].

A system designed using the state-space approach consists of process component (characterized using Eq. (1)) and observation component (characterized using Eq. (2)) [10]. The internal system behavior (depicted using system state) along with the associated noise (error) and the system input vector is represented using the process component; and the system status along with the status evaluation noise (error) and the system input vector is evaluated using the system state by the observation component.

The system designed using the state-space approach is represented as [10] -

$$\varphi_{i+1} = X\varphi_i + P\varpi_i + Q\eta_i \tag{1}$$

$$o_i = Z\varphi_i + R\varpi_i + N\zeta_i \tag{2}$$

Where,

i: time instant
φ_i : system state vector of dimension $(a \times 1)$, consisting of 'a' state variables
o_i : system status vector of dimension $(b \times 1)$, consisting of 'b' status variables
ϖ_i : system input vector of dimension $(e \times 1)$, consisting of 'e' input variables
η_i, ζ_i : stochastic error vector of dimension $(c \times 1)$, $(d \times 1)$, respectively
X, P, Z, R, Q, N : system matrices of dimension $(a \times a)$, $(a \times e)$, $(b \times a)$, $(b \times e)$, $(a \times c)$, $(b \times d)$, respectively

4 Design Issues

This section consists of the design issues used for the research.

4.1 Agile Software Development Philosophy

The software products are developed by organizations. Generally, organizations have to face a problem: what type of product is required to be developed (i.e., determining the product scope). In order to tackle the problem, organizations follow agile software development philosophy.

According to agile software development philosophy ([15], [16]) the product is developed in such a way that accepts the uncertainty associated with the product scope. In order to deal with the uncertainty, the product is evolved iteratively using the feedback principle for developing the software product.

4.2 Fine Grain Development Approach

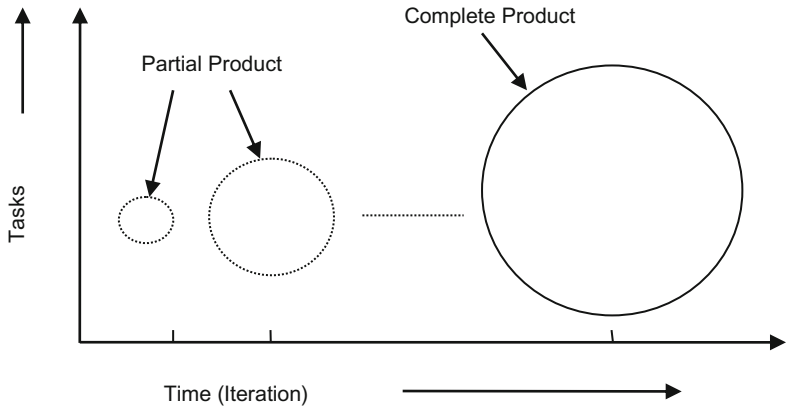


Fig. 1. Fine grain development

According to the fine grain development [8], the product is evolved iteratively by performing an up-front analysis phase briefly (determining the tasks for the product development, i.e., list of tasks) for an iteration. The complete product and the partial versions, as shown in Fig. 1, are paraded as the thick circle and the dotted circles, respectively. An increment development time is entitled as iteration. Iterations are paraded by marking on time axis.

4.3 Partially Completed Tasks

Software products are evolved through development activities: requirements analysis, designing, coding, testing and integrating. The product development work is represented in terms of the tasks [17]. These tasks are needed to be completed as to develop

the product. In order to generalize the technique, tasks may refer to a module consisting of some lines-of-code, function-points; user-stories (size metric is story-points); or features (size metric is feature-points) et-cetera.

In this paper, we consider a task referring a module consisting of some function-points. During iteration, tasks are implemented by following the activities: designing, coding, testing and integrating. At the time of project status reporting, few of the tasks are discharged; partially completed; and untouched. A partially completed task is one whose implementation has been instigated but not yet concluded. The amount of work has been completed related to a partially completed task for the project status reporting is assessed by an expert.

4.4 Project Planning and Execution

Iteration-wise project planning and execution is performed. Following factors: team productivity, organizational productivity, and requirements volatility are taken into consideration for the project planning. Team productivity means the tasks completed during iteration [18]. Organizational productivity means tasks completed per effort applied (person-iteration) [19].

The baseline values for different technologies for team productivity and organizational productivity derived from a database of past projects are used to carry out the future projects [20]. In case of a project for which there is no specific technology baseline value exists then considers the similar technology baseline value. For example, for C++, if baseline value not available, we can consider the baseline value for C. If similar technology baseline value is not available then consider the average of all the existing baseline values as the languages are correlated to each other.

As per the team productivity, the tasks are planned for implementing during iteration. The organizational productivity is used for predicting the effort desired for project completion during the planning and the execution. During the planning, the baseline value is used for the organizational productivity. During the execution, the organizational productivity is evaluated as: $\mu \times$ organizational productivity during the planning + $(1 - \mu)\times$(tasks concluded during iteration)/effort consumed, where 'μ' is considered to be in between 0 and 1, inclusively and the symbols $\times$, +, −,/ represent the mathematical operations multiplication, addition, subtraction, division, respectively. The tasks concluded and the effort consumed during iteration is collected as the project is executed. Based on the value given to the past or present performance of the team, the 'μ' is assigned a value.

The project is planned by considering the tasks changes due to the requirements volatility phenomenon. The change related to the tasks during the product development is termed as requirements volatility [21] and the changes are the outcome of the requirements volatility phenomenon. We assume that the tasks planned for implementation during an iteration and the tasks which have been already implemented during the previous iterations are unaffected by the requirements volatility phenomenon. It means we would not consider the changes related to them during the project planning.

According to the fine grain development approach, a list of tasks is maintained throughout the product development. For project planning, three types of tasks changes

adjured by the customers are taken into account ([22]): (i) new tasks to be added-on to the list of tasks, (ii) tasks to be expunged from the list of tasks, and (iii) tasks spawned owing to the refinement of the tasks related to the list of tasks.

During the analysis phase of iteration, the customers are asked for the new tasks to be added-on to the list of tasks and the tasks to be expunged from the list of tasks during planning and execution. During the project planning, the parameter 'δ' (measured in terms of percentage) is used to control the tasks spawned owing to the refinement of the tasks related to the list of tasks and the tasks spawned are gauged by multiplying the size of the tasks related to the list of tasks with 'δ'; while during the project execution, the tasks spawned are collected. Assume that the baseline value for the parameter 'δ' derived from a database of past projects exists and is used to carry out the future projects.

5 Tracking Technique Formulation Using State-Space Approach

Software projects are planned and executed. These are tracked by the evaluation of their execution status as compared to planned status. Their execution and planned status are procured using their execution and planned state, respectively. The project state and the project status are depicted by the state vector and the status vector, respectively.

According to state-space approach, the tracking technique is formulated by the formulation of the process component exercising Eq. (1) and the observation component exercising Eq. (2) during the project planning and the execution. The process component and the observation component are utilized to decide the project state and the project status, respectively.

5.1 Plan Space

Let A_p:$[A1_p, A2_p, A3_p, A4_p]^T$ stands for a project state vector of (4 × 1) and E_p:$[C_p, O_p, O_p]^T$ stands for an external input vector of (3 × 1). The external input vector controls the project transition from a state to other. Let B_p:$[rmrq_p, efrm_p, tmst_p, pjvl_p,]^T$ stands for a project status vector of (4 × 1). Where the symbols 'p' and 'T' represent the end of the pth iteration and matrix transpose operation, respectively. The variables related to the aforementioned vectors are assigned the following denotations:

$A1_p$: Tasks changes adjured by the customers for the pth iteration

$A2_p$: Tasks winded-up for the pth iteration

$A3_p$: Tasks to be winded-up for ending the project at the pth iterations' tail end

$A4_p$: Tasks to be winded-up for ending the project at the pth iterations' head end

C_p: Control variable related to tasks changes (new tasks to be added-on to the list of tasks, and tasks to be expunged from the list of tasks) adjured by the customers for p^{th} iteration

O_p: Tasks winded-up for the pth iteration

U_p: Tasks spawned owing to the refinement of the tasks related to the list of tasks for the pth iteration

$rmrq_p$: Tasks to be winded up for ending the project at the pth iterations' tail end
$efrm_p$: Effort expected for ending the project at the pth iterations' tail end
$tmst_p$: Team strength (rate of tasks winded-up by the team) for the pth iteration
$pjvl_p$: Project velocity (rate of project cessation) for the pth iteration
p: End of the pth iteration

For $(p + 1)^{th}$ iteration, Fig. 2 renders the project state transformation from A_p to A_{p+1} owing to external input vector E_p. The project state vector A_{p+1} is computed as:

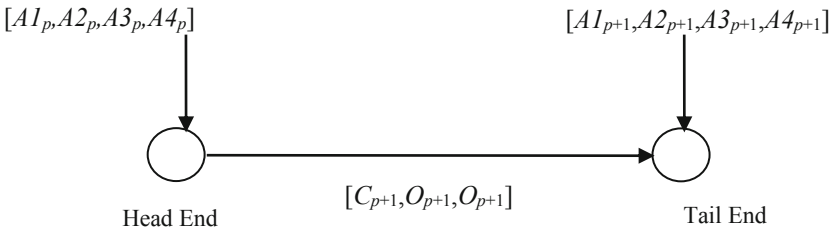


Fig. 2. Project state transformation through $(p+1)^{th}$ iteration

Assessment of $A1_{p+1}$: Throughout $(p+1)^{th}$ iteration, new tasks to be added-on to the list of tasks, and tasks to be expunged from the list of tasks adjured by the customers are collected as to assess the C_{p+1} which is figured out as

C_{p+1} = new tasks to be added-on to the list of tasks − tasks to be expunged from the list of tasks; where the symbol '−' represents the mathematical operation subtraction

The meaning associated to the project state variable '$A1_{p+1}$' and the external input variable 'C_{p+1}' implies both are similar.

Therefore, $A1_{p+1} = C_{p+1}$

Assessment of $A2_{p+1}$: Throughout $(p+1)^{th}$ iteration, Tasks winded-up are alluded by O_{p+1}. The meaning associated to the project state variable '$A2_{p+1}$' and the external input variable 'O_{p+1}' implies both are similar.

Therefore, $A2_{p+1} = O_{p+1}$

Assessment of $A3_{p+1}$: $A3_{p+1}$ is assessed by adding the C_{p+1}, $(A3_p - O_{p+1})$, and U_{p+1}.

Therefore, $A3_{p+1} = C_{p+1} + (A3_p - O_{p+1}) + U_{p+1}$

$= C_{p+1} + (A3_p - O_{p+1}) + \delta \times (A3_p - O_{p+1})$; as $U_{p+1} = \delta \times (A3_p - O_{p+1})$ and δ is a controlling parameter for U_{p+1}

$= C_{p+1} + (\delta + 1) \times A3_p - (\delta + 1) \times O_{p+1}$

Assessment of $A4_{p+1}$: The meaning associated to the project state variable '$A4_{p+1}$' and '$A3_p$' implies both are similar.

Therefore, $A4_{p+1} = A3_p$

By arranging the state variables, these are as follows

$A1_{p+1} = 0 \times A1_p + 0 \times A2_p + 0 \times A3_p + 0 \times A4_p + 1 \times C_{p+1} + 0 \times O_{p+1} + 0 \times O_{p+1}$

$A2_{p+1} = 0 \times A1_p + 0 \times A2_p + 0 \times A3_p + 0 \times A4_p + 0 \times C_{p+1} + 1 \times O_{p+1} + 0 \times O_{p+1}$

$A3_{p+1} = 0 \times A1_p + 0 \times A2_p + (\delta + 1) \times A3_p + 0 \times A4_p + 1 \times C_{p+1} - \delta \times O_{p+1} - 1 \times O_{p+1}$

$A4_{p+1} = 0 \times A1_p + 0 \times A2_p + 1 \times A3_p + 0 \times A4_p + 0 \times C_{p+1} + 0 \times O_{p+1} + 0 \times O_{p+1}$

Process Component

$$\begin{bmatrix} A1_{p+1} \\ A2_{p+1} \\ A3_{p+1} \\ A4_{p+1} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & (\delta+1) & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} A1_p \\ A2_p \\ A3p \\ A4p \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & -\delta & -1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} C_{p+1} \\ O_{p+1} \\ O_{p+1} \end{bmatrix} \tag{3}$$

At moment 'p', the project status vector $Bp{:}[rmrqp, efrmp, tmstp, pjvlp]^T$ is computed with the project state vector $Ap{:}[A1p, A2p, A3p, A4p]^T$. According to the meaning associated with the status variables and the state variables, the project status vector is computed as follows:

$rmrq_p = A3_p$

$efrm_p = \alpha \times A3_p$, where α (quantified in terms of person-iteration/tasks) be the multiplicative inverse of the organizational productivity during the planning and alludes to a multiplicative factor for turning $A3_p$ to $efrm_p$

$tmst_p = A2_p$

$pjvl_p = A4_p - A3_p$

By arranging the status variables, these are as follows

$rmrq_p = 0 \times A1_p + 0 \times A2_p + 1 \times A3_p + 0 \times A4_p$

$pjvl_p = 0 \times A1_p + 0 \times A2_p - 1 \times A3_p + 1 \times A4_p$

$efrm_p = 0 \times A1_p + 0 \times A2_p + \alpha \times A3_p + 0 \times A4_p$

$tmst_p = 0 \times A1_p + 1 \times A2_p + 0 \times A3_p + 0 \times A4_p$

Observation Component

$$\begin{bmatrix} rmrq_p \\ efrm_p \\ tmst_p \\ pjvl_p \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & \alpha & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} A1_p \\ A2_p \\ A3_p \\ A4_p \end{bmatrix} \tag{4}$$

5.2 Execution Space

Let $A'_p{:}[A1'_p, A2'_p, A3'_p, A4'_p]^T$ stands for a project state vector of (4×1) and $E'_p{:}[C_p, O'_p, U'_p]^T$ stands for an external input vector of (3×1). The external input vector controls the project transition from a state to other. Let $B'_p{:}[rmrq'_p, efrm'_p, tmst'_p, pjvl'_p,]^T$ stands for a project status vector of (4×1). Where the symbols 'p' and 'T' represent the end of the pth iteration and matrix transpose operation, respectively. The variables related to the aforementioned vectors are assigned the following denotations:

$A1'_p$: Tasks changes adjured by the customers for the pth iteration

$A2'_p$: Tasks winded-up and partially completed for the pth iteration

$A3'_p$: Tasks to be winded-up for ending the project at the pth iterations' tail end

$A4'_p$: Tasks to be winded-up for ending the project at the pth iterations' head end

C_p: Control variable related to tasks changes (new tasks to be added-on to the list of tasks, and tasks to be expunged from the list of tasks) adjured by the customers for p^{th} iteration

O'_p: Tasks winded-up and partially completed for the pth iteration

U'_p: Tasks spawned owing to the refinement of the tasks related to the list of tasks for the pth iteration

$rmrq'_p$: Tasks to be winded up for ending the project at the pth iterations' tail end

$efrm'_p$: Effort expected for ending the project at the pth iterations' tail end

$tmst'_p$: Team strength (rate of tasks winded-up by the team) for the pth iteration

$pjvl'_p$: Project velocity (rate of project cessation) for the pth iteration

p: End of pth iteration

For $(p+1)^{th}$ iteration, Fig. 3 renders the project state transformation from A'_p to A'_{p+1} owing to external input vector E'_p. The project state vector A'_{p+1} is computed as:

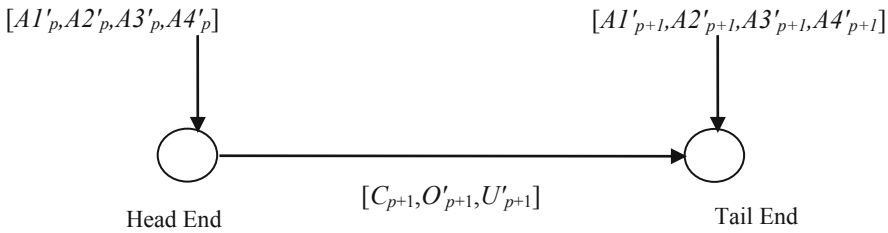


Fig. 3. Project state transformation through $(p+1)^{th}$ iteration

Assessment of $A1'_{p+1}$: Throughout $(p+1)^{th}$ iteration, new tasks to be added-on to the list of tasks, and tasks to be expunged from the list of tasks adjured by the customers are collected as to assess the C_{p+1} which is figured out as

C_{p+1} = new tasks to be added-on to the list of tasks − tasks to be expunged from the list of tasks; where the symbol '−' represents the mathematical operation subtraction

The meaning associated to the project state variable '$A1'_{p+1}$' and the external input variable 'C_{p+1}' implies both are similar.

Therefore, $A1'_{p+1} = C_{p+1}$

Assessment of $A2'_{p+1}$: Throughout $(p + 1)^{th}$ iteration, Tasks winded-up are alluded by O'_{p+1}. The meaning associated to the project state variable '$A2'_{p+1}$' and the external input variable 'O'_{p+1}' implies both are similar.

Therefore, $A2'_{p+1} = O'_{p+1}$

Assessment of $A3'_{p+1}$: $A3'_{p+1}$ is assessed by adding the C_{p+1}, $(A3'_p - O'_{p+1})$, and U'_{p+1}.

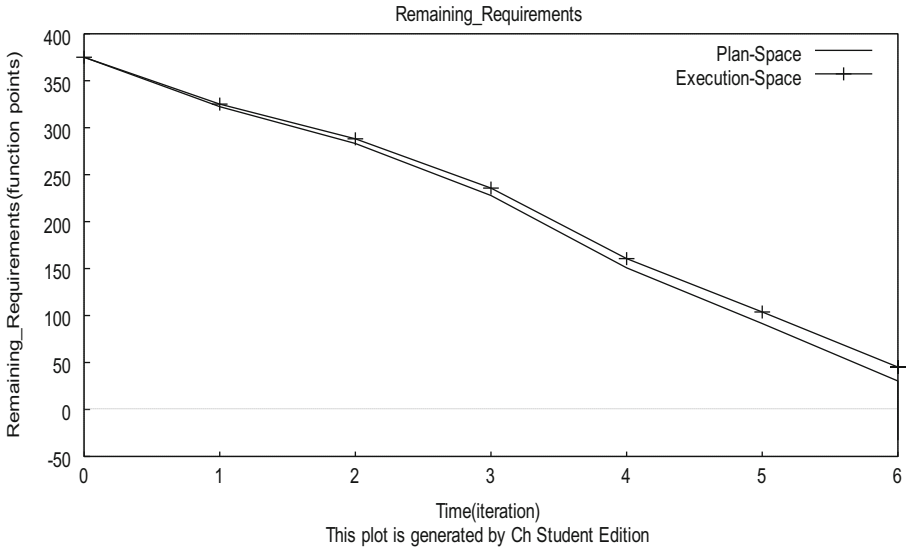
Therefore, $A3'_{p+1} = C_{p+1} + (A3'_p - O'_{p+1}) + U'_{p+1}$

Assessment of $A4'_{p+1}$: The meaning associated to the project state variable '$A4'_{p+1}$' and '$A3'_p$' implies both are similar.

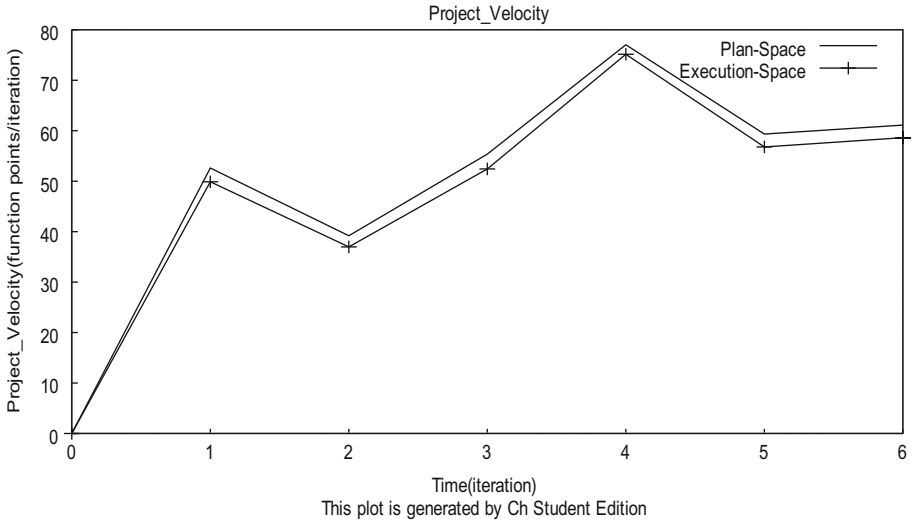
Therefore, $A4'_{p+1} = A3'_p$

By arranging the state variables, these are as follows

$A1'_{p+1} = 0 \times A1'_p + 0 \times A2'_p + 0 \times A3'_p + 0 \times A4'_p + 1 \times C_{p+1} + 0 \times O'_{p+1} + 0 \times U'_{p+1}$

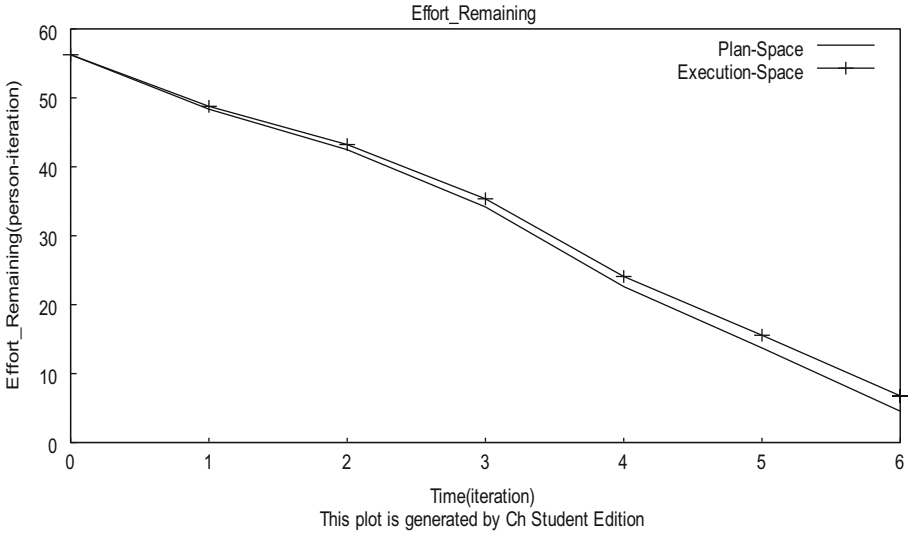


(a)

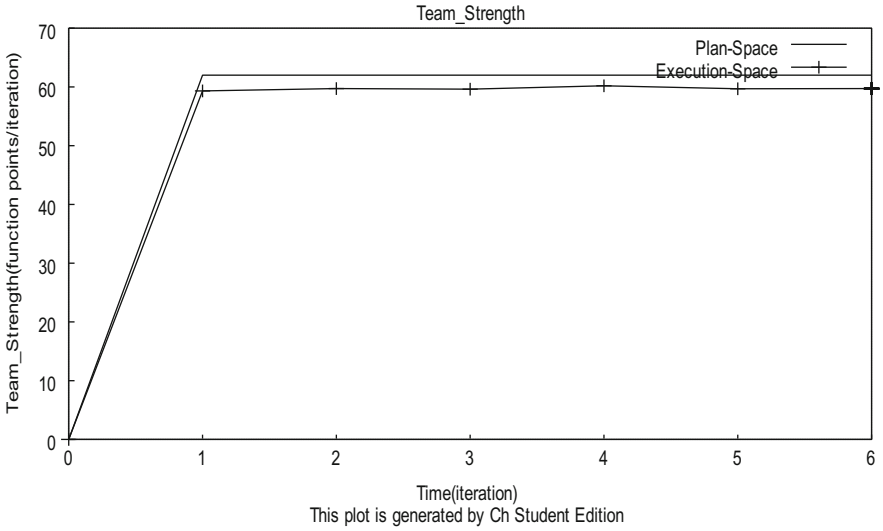


(b)

Fig. 4. Agile project status at the end of iteration



(c)



(d)

Fig. 4. (*continued*)

$A2'_{p+1} = 0 \times A1'_p + 0 \times A2'_p + 0 \times A3'_p + 0 \times A4'_p + 0 \times C_{p+1} + 1 \times O'_{p+1} + 0 \times U'_{p+1}$

$A3'_{p+1} = 0 \times A1'p + 0 \times A2'_p + 1 \times A3'_p + 0 \times A4'_p + 1 \times C_{p+1} - 1 \times O'_{p+1} + 1 \times U'_{p+1}$

$A4'_{p+1} = 0 \times A1'_p + 0 \times A2'_p + 1 \times A3'_p + 0 \times A4'_p + 0 \times C_{p+1} + 0 \times O'_{p+1} + 0 \times U'_{p+1}$

$$\text{Process Component}$$

$$\begin{bmatrix} A1'_{p+1} \\ A2'_{p+1} \\ A3'_{p+1} \\ A4'_{p+1} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} A1'_p \\ A2'_p \\ A3'p \\ A4'p \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & -1 & -1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} C_{p+1} \\ O'_{p+1} \\ U'_{p+1} \end{bmatrix} \tag{5}$$

At moment 'p', the project status vector B'_p:$[rmrq'_p, efrm'_p, tmst'_p, pjvl'_p]^T$ is computed with the project state vector A'_p:$[A1'_p, A2'_p, A3'_p, A4'_p]^T$. According to the meaning associated with the status variables and the state variables, the project status vector is computed as follows:

$rmrq'_p = A3'_p$

$efrm'_p = \alpha' \times A3'_p$, where α' (quantified in terms of person-iteration/tasks) be the multiplicative inverse of the organizational productivity during the execution and alludes to a multiplicative factor for turning $A3'_p$ to $efrm'_p$

$tmst'_p = A2'_p$

$pjvl'_p = A4'_p - A3'_p$

By arranging the status variables, these are as follows

$rmrq'_p = 0 \times A1'_p + 0 \times A2'_p + 1 \times A3'_p + 0 \times A4'_p$

$pjvl'_p = 0 \times A1'_p + 0 \times A2'_p - 1 \times A3'_p + 1 \times A4'_p$

$efrm'_p = 0 \times A1'_p + 0 \times A2'_p + \alpha' \times A3'_p + 0 \times A4'_p$

$tmst'_p = 0 \times A1'_p + 1 \times A2'_p + 0 \times A3'_p + 0 \times A4'_p$

$$\text{Observation Component}$$

$$\begin{bmatrix} rmrq'_p \\ efrm'_p \\ tmst'_p \\ pjvl'_p \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & \alpha' & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} A1'_p \\ A2'_p \\ A3'_p \\ A4'_p \end{bmatrix} \tag{6}$$

Equations (**3**) and (4) (Eqs. (5) and (6)) are utilized to report the project status during planning and execution.

6 Simulation

The project "Development of a Software Package for a Computer-Aided Design Application for Mechanical Components" depicted in [23] is considered as to show the practicability of the tracking technique. The project is utilized for initializing the technique and the results of tracking (project status) acquired by utilizing Monte-Carlo simulation are reported during planning and execution. The Ch Interpreter (C/C++ Interpreter), a tool developed by Dr. Harry H. Chang, is exercised to conduct the simulation of the technique.

Table 1. Agile project status at the end of iteration

Plan space				
Iteration#	*rmrq* (function points)	*pjvl* (function points/iteration)	*efrm* (person-iteration)	*tmst* (function points/iteration)
0	375.000000	0.000000	56.250000	0.000000
1	322.390000	52.610000	48.358500	62.000000
2	283.201700	39.188300	42.480255	62.000000
3	227.837751	55.363949	34.175663	62.000000
4	150.812884	77.024867	22.621933	62.000000
5	91.477270	59.335613	13.721591	62.000000
6	30.361588	61.115682	4.554238	62.000000
Execution space				
Iteration#	*rmrq′* (function points)	*pjvl′* (function points/iteration)	*efrm′* (person-iteration)	*tmst′* (function points/iteration)
0	375.000000	0.000000	56.250000	0.000000
1	325.124190	49.875810	48.768629	59.324317
2	288.151145	36.973046	43.222672	59.720795
3	235.724340	52.426805	35.358651	59.616420
4	160.580129	75.144212	24.087019	60.192045
5	103.797943	56.782185	15.569692	59.691223
6	45.181825	58.616119	6.777274	59.719447

6.1 Simulation Study

The project is started with the tasks consisting of 375 function points. Thus the initial project state during planning and execution are A_0: $[0, 0, 375, 375]^T$ and A'_0: $[0, 0, 375, 375]^T$, respectively. The external input vector during planning and execution are E_0: $[0, 0, 0]^T$ and E'_0: $[0, 0, 0]^T$, respectively. The baseline values for team productivity and organizational productivity are 62 function points/iteration and 6.5 function points/person-iteration, respectively. Based on the team productivity, the tasks to be implemented are planned through iteration and their implementation is started. The organizational productivity is utilized for transforming the tasks to effort (measured in terms of person-iteration). The value of 'α' is 1/6.5 person-iteration/function points during planning and the value of 'α''' is 1/6.5 person-iteration/function points assuming 'μ' is 1. The parameter 'δ' is 3%. During the 2nd iteration, tasks of 15 function points are added to the list of tasks and during the 4th iteration, tasks of 20 function points are removed from the list of tasks.

6.2 Results

The results of tracking the project are paraded both quantitatively in the tabular form and graphically as the curves against the status variables *rmrq*, *pjvl*, *efrm*, and *tmst*

(*rmrq′*, *pjvl′*, *efrm′*, and *tmst′*), during the planning (execution). The curves against the status variables *rmrq*, *pjvl*, *efrm*, and *tmst* (*rmrq′*, *pjvl′*, *efrm′*, and *tmst′*) are titled as Remaining_Requirements, Project_Velocity, Effort_Remaining, and Team_Strength, respectively.

The project status is paraded at the end of iteration. For example, at the 2nd iterations' tail end, Tasks to be winded up for ending the project (designated with *rmrq* and *rmrq′*) as paraded in Fig. 4(a) are in excess through execution as compared to planning, Project velocity (designated with *pjvl* and *pjvl′*) as paraded in Fig. 4(b) is lowly through execution as compared to planning, Effort expected for ending the project (designated with *efrm* and *efrm′*) as paraded in Fig. 4(c) is in excess through execution as compared to planning, and Team strength (designated with *tmst* and *tmst′*) as paraded in Fig. 4(d) is lowly through execution as compared to planning. The project status is paraded quantitatively in Table 1.

The impact of agility is also shown over the project status. Observe that at the 2nd iterations' tail end, Tasks to be winded up for ending the project (designated with *rmrq* and *rmrq′*) as paraded in Fig. 4(a) should be decreased rather than increased as contrasted with the 1st iterations' tail end. This is because of customer's behest for appending the tasks of 15 function points to the list of tasks during the 2nd iteration. As a result of this, the Project velocity (designated with *pjvl* and *pjvl′*) as paraded in Fig. 4 (b) falls down sharply; the Effort expected for ending the project (designated with *efrm* and *efrm′*) as paraded in Fig. 4(c) moves up rather than down. Thereafter, at the 4th iterations' tail end, Tasks to be winded up for ending the project (designated with *rmrq* and *rmrq′*) as paraded in Fig. 4(a) falls down sharply as contrasted with the 3rd iterations' tail end. This is because of customer's behest for eradicating the tasks of 20 function points from the list of tasks during the 4th. As a result of this, the Project velocity (designated with *pjvl* and *pjvl′*) as paraded in Fig. 4(b) moves up sharply; the Effort expected for ending the project (designated with *efrm* and *efrm′*) as paraded in Fig. 4(c) moves down sharply as contrasted with the 3rd iterations' tail end.

7 Conclusion and Future Work

Heavy-weight tracking techniques have been developed by mainly focusing on the better project visibility (maintaining the state information of the partially completed tasks) to decide some actions, but does not make a balance between the benefits of the project visibility and the resources required for maintaining it. In this paper, a light-weight tracking technique has been developed which considers not only the completed tasks but also the partially completed tasks for maintaining the project visibility; but it does not maintain the state information of the partially completed tasks, which makes a tracking technique as heavy-weight. The amount of the work completed related to the partially completed tasks is estimated by an expert.

The state-space approach is employed to develop the technique. According to state-space approach, the technique is made up of the process component and the observation component in the plan space and the execution space. The technique reports explicitly the projects' execution status as compared to the planned status. The software development organizations need to maintain the baseline values for team productivity,

organizational productivity, and the parameter used for controlling the tasks spawned owing to the refinement of the tasks related to the list of tasks. The baseline values are employed for planning the project.

The technique can be further improved by incorporating the uncertainties of the software development environment associated with the project planning and the execution. Rather than using the expert based estimation for the amount of work completed related to partially completed tasks, the technique can be designed for estimating it. A simulated project has been used as to validate the technique; this can be an issue for the future research work.

References

1. Project Management Institute: Practice standard for earned value management. 2nd edn. Project Management Institute Inc., Four Campus Boulevard, Newton Square, Pennsylvania, 19073–3299 USA (2006)
2. GAO. Cost Assessment Guide: Best Practices for estimating and managing program costs. United States Government Accountability Office, Washington, US Government, GAO-07-1134SP (2007)
3. Schwaber, K., Hundhausen, R., Starr, D.: Agile Project Management with Scrum, 2nd edn. Microsoft Press, Redmond (2015)
4. Anderson, D.J.: Agile Management for Software Engineering: Applying the Theory of Constraints for Business Results. Prentice-Hall PTRm, New Jersey (2004)
5. Anderson, D.J.: Using cumulative flow diagrams. The Coad Letter – Agile Management. Prentice-Hall PTRm, New Jersey (2004)
6. Microsoft: https://www.microsoft.com/en-us/download/confirmation.aspx?id=5365. Accessed 20 Sept 2020
7. Wake, W.C.: Extreme Programming Explored, 1st edn. Addison-Wesley Professional, Boston (2001)
8. Miranda, E., Bourque, P.: Agile monitoring using the line of balance. The J. of Sys. Soft. **83**, 1205–1215 (2010)
9. Agile Manifesto: https://agilemanifesto.org/. Accessed 20 Sept 2020
10. Burns, R.S.: Advanced control engineering, 1st edn. Butterworth-Heinemann Elsevier, Amsterdam (2001)
11. Cohn, M.: Agile Estimating and Planning. Prentice-Hall, India (2006)
12. Jackson, P.: The cumulative flow plot: understanding basic concepts in material flow. Tijdschrift voor Econornie en Management XXXVI (3) (1991)
13. Rusk, J.: Earned Value for Agile Development. Optimation Ltd. (2009)
14. Cabri, A., Griffiths, M.: Earned value and agile reporting. IEEE Comp. Soc. 17–22 (2006)
15. Boehm, B.: Get ready for agile methods, with care. IEEE Comp. **35**(1), 64–69 (2002)
16. Dyba, T.: Improvisation in small software organizations. IEEE Soft. **17**(5), 82–87 (2000)
17. Boehm, B.W.: Software Engineering Economics. Prentice- Hall, Englewood Cliffs, NJ (1981)
18. Rodríguez, D., Sicilia, M.A., Garcíaa, E., Harrison, R.: Empirical findings on team size and productivity in software development. J. Sys. Soft. **85**, 562–570 (2012)
19. Mills, H.D.: Software Productivity. Little, Brown & Co., Canada (1983)
20. www.oreilly.com view practical-software-estimation. Accessed 02 Aug 2020

21. Pfahland, D., Lebsanft, K.: Using simulation to analyse the impact of software requirement volatility on project performance. Info. Soft. Tech. **42**, 1001–1008 (2000)
22. Costello, R.J., Liu, D.: Metrics for requirements engineering. J. of Sys. and Soft. **29**, 39–63 (1995)
23. Pressman, R.S.: Software Engineering a Practitioner's Approach, 6th edn. New Delhi, McGraw Hill Education (India) Private Limited (2010)